Contenedores comunicados con Docker y pipeline de GitHub Actions

Entrega 1 – Aplicación Flask con PostgreSQL orquestada por Docker Compose

Integrante

Nombre	Correo / usuario	
Miguel Angel Aguilar Luna	**SavitarADSO** pee7491@gmail.com	–

Módulo	Énfasis Profesional I (Integración Continua)
Docente	Edward Reyes Corredor
Fecha	10 de noviembre de 2026
Repositorio	https://github.com/SavitarADSO/-nfasis-Profesional-I-Integraci-n-Continua-

Documento generado con LaTeX | Proyecto de software basado en herramientas de integración continua | Docker + Docker Compose + GitHub Actions

Índice

1. Resumen **4**

2. Introducción **4**
2.1. Contexto 4
2.2. Objetivo general 4
2.3. Objetivos específicos 4
2.4. Alcance y estructura de las entregas 5

3. Marco teórico **5**
3.1. Contenedores y flujo de construcción de una imagen 5
3.2. Orquestación con Docker Compose 6
3.3. Redes de Docker y resolución de nombres 6
3.4. Integración continua con GitHub Actions 6
3.5. Gestión de secretos 7

4. Metodología **7**
4.1. Estructura del repositorio 7
4.2. Diagrama de arquitectura 8
4.3. Flujo de datos de una operación 8

5. Implementación: desarrollo de los ocho pasos **8**
5.1. Paso 1. Repositorio de Git con commits frecuentes 8
5.2. Paso 2. Dockerfile con imagen base de versión fija 10
5.3. Paso 3. Orquestación con Docker Compose 10
5.4. Paso 4. Aplicación Flask y base de datos 11
5.5. Paso 5. Comunicación entre contenedores por nombre de servicio 12
5.6. Paso 6. Gestión de secretos: `.env` y `.env.example` 13
5.7. Paso 7. Pipeline de integración continua en GitHub Actions 14
5.8. Paso 8. Informe final en PDF 15

6. Pruebas y resultados **15**
6.1. Caso de prueba de extremo a extremo 16
6.2. Evidencia gráfica del entorno 16

7. Solución de problemas **18**

8. Conclusiones **18**

Referencias **19**

A. Apéndice A. Código completo de la aplicación **21**

B. Apéndice B. Guía rápida de puesta en marcha **23**

C. Apéndice C. Lista de capturas requeridas **24**

Índice de figuras

1. Arquitectura de la solución: la aplicación Flask (contenedor `app`) accede a PostgreSQL (contenedor `db`) a través de la red propia de Compose usando el nombre de servicio `db`; los datos persisten en el volumen `db_data`. 8
2. Historial de commits del repositorio en GitHub, donde se observa la secuencia de avances independientes. 9
3. Escritura de un mensaje en la base de datos a través de la API (`POST /mensajes`); la respuesta incluye el `id` asignado por PostgreSQL. 12
4. Lectura de los mensajes almacenados (`GET /mensajes`); el dato ingresado en la figura anterior aparece en la respuesta. 13
5. Resolución del nombre de servicio `db` desde dentro del contenedor `app`; la dirección devuelta (`172.18.0.2`) es la IP interna del contenedor de base de datos dentro de `red_proyecto`. 13
6. Consulta `SELECT` ejecutada directamente en PostgreSQL, que confirma la persistencia del mensaje en el volumen `db_data`. 13
7. Ejecución exitosa del pipeline en la pestaña *Actions* del repositorio: la corrida `Docker CI` aparece en verde y los seis pasos —copiar el código, crear el `.env` de prueba, construir las imágenes, levantar y probar la comunicación entre contenedores y apagar el entorno— aparecen con conclusión `success`. 15
8. Construcción de las imágenes con `docker compose build -no-cache`; se observa la exportación de las capas y la creación de la imagen `proyecto-ci-app`. 16
9. Levantamiento de los dos contenedores con `docker compose up -d`; el log muestra la creación del volumen `db_data`, de la red `red_proyecto` y el arranque ordenado de `ci_db` (healthy) y `ci_app`. 17
10. Salida de `docker ps` con los contenedores `ci_app` y `ci_db` en estado *Up (healthy)*, y sus puertos e identificadores. 17
11. Salida de `docker network inspect`: los dos contenedores pertenecen a la misma red `red_proyecto` y cada uno tiene asignada una dirección IP interna, que es la que resuelve el nombre de servicio desde la app. . . . 17
12. Logs del contenedor `app`: se observa que la base de datos está disponible desde el primer intento, que la tabla `mensajes` se crea al arrancar y que Flask escucha en `0.0.0.0:5000`, dirección necesaria para que el contenedor sea alcanzable desde fuera. 18

1. Resumen

Este informe documenta la **Entrega 1** del proyecto de integración continua de la materia *Integración Continua y Entrega Continua (CI/CD)*. La entrega consiste en construir una aplicación web sencilla que se ejecuta dentro de un contenedor Docker y que almacena y consulta información en una base de datos PostgreSQL alojada en un segundo contenedor. Ambos contenedores se definen, configuran y levantan de manera reproducible mediante un archivo `docker-compose.yml`, y la automatización de la construcción y las pruebas se realiza con un pipeline de *GitHub Actions*.

El punto técnico central de la entrega es la **comunicación entre contenedores por nombre de servicio**. En lugar de direccionar la aplicación hacia `localhost` o hacia una dirección IP fija, el código resuelve el host `db`, que es el nombre de servicio declarado en Docker Compose. Esta decisión hace que la solución sea independiente de la topologáá de la red, portable entre distintos host y coherente con el modelo declarativo de Compose. Además, se implementó una gestión responsable de secretos mediante un archivo `.env` excluido del control de versiones y una plantilla versionada `.env.example`.

Se concludes que la automatización del entorno de ejecución es tan importante como la automatización del código: el pipeline construido en esta entrega construye las imágenes, levanta el stack completo, verifica de extremo a extremo que la aplicación guarda y lee datos de la base de datos y desmonta el entorno, garantizando que cada commit entregue sea ejecutable.

Palabras clave: Docker, Docker Compose, contenedores, PostgreSQL, Flask, integración continua, GitHub Actions, redes bridge, gestión de secretos.

2. Introducción

2.1 Contexto

El desarrollo de software moderno exige entregas frecuentes y reproducibles. Un equipo que depende de que cada miembro instale manualmente las dependencias, configure la base de datos y pruebe a mano no puede sostener un ritmo de entrega semanal. Docker responde a esta problemática empaquetando la aplicación y todas sus dependencias en una imagen aislada y ligera, mientras que Docker Compose describe en un solo archivo declarativo cómo varios contenedores deben relacionarse entre sí.

2.2 Objetivo general

Implementar un sistema de dos contenedores comunicados –una API Flask y una base de datos PostgreSQL– orquestados con Docker Compose, con la comunicación resuelta por nombre de servicio, y automatizar su construcción y verificación mediante un pipeline de integración continua en GitHub Actions.

2.3 Objetivos específicos

1. Crear un repositorio de Git con una estructura clara y con *commits* frecuentes que documenten cada avance.

2. Construir la imagen de la aplicación mediante un `Dockerfile` que use una imagen base con versión fija.
3. Definir los dos servicios, la red propia, el volumen persistente y la condición de dependencia en un `docker-compose.yml`.
4. Implementar los endpoints de escritura y lectura sobre PostgreSQL.
5. Demostrar experimentalmente que la aplicación se conecta a la base de datos usando el nombre de servicio `db` y no `localhost`.
6. Gestionar los secretos con `.env` (excluido) y `.env.example` (versionado).
7. Configurar GitHub Actions para construir, probar y desmontar el stack en cada *push* o *pull request*.
8. Entregar el presente informe en formato PDF con evidencia gráfica.

2.4 Alcance y estructura de las entregas

El proyecto se desarrolla en tres entregas sobre el mismo repositorio:

Entrega	Semana	Actividad	Incidencia en el repositorio
1	3	Dos contenedores comunicados entre sí	App Flask, PostgreSQL, `docker-compose.yml`, workflow de GitHub Actions y este informe.
2	5	Jenkins como servidor de integración continua	Pipeline de Jenkins y documento de requisitos para su implementación.
3 y sustentación	7–8	Todo integrado (contenedores, Jenkins, Travis CI y Codeship)	Documento final con historial de cambios, solución de problemas, responsabilidades y opiniones.

Este documento cubre **exclusivamente la Entrega 1**.

3. Marco teórico

3.1 Contenedores y flujo de construcción de una imagen

Un contenedor es una instancia ejecutable de una imagen, aislada del sistema anfitrión mediante tecnologías denamespaces y cgroups del núcleo Linux (Docker Desktop lo implementa sobre una máquina virtual ligera en Windows y macOS). La ventaja para el proyecto no es la "eficiencia" del contenedor en sí, sino la **reproducibilidad**: la imagen encapsula el sistema operativo mínimo, el intérprete y las dependencias exactas del proyecto.

La construcción de una imagen se define en un `Dockerfile`, un archivo declarativo de instrucciones. Cada instrucción genera una capa (layer) que se cachea: si una capa

no cambia, Docker reutiliza la que ya tiene. De ahí la práctica de copiar primero el archivo de dependencias y ejecutar `pip install` antes de copiar el código fuente, de modo que una modificación en `app.py` no invalide la capa de dependencias.

3.2 Orquestación con Docker Compose

Docker Compose es una herramienta que declara un conjunto de servicios en un archivo YAML y los ejecuta de forma conjunta. En lugar de una secuencia de comandos `docker run` complicada de reproducir, se declara el estado deseado y Compose lo alcanza. Entre los elementos usados en esta entrega se encuentran:

- **Servicios**: cada contenedor del sistema. En este proyecto, `app` (construida desde el `Dockerfile`) y `db` (imagen oficial `postgres:16.4`).
- **Nombres de servicio**: alias DNS internos de la red de Compose. Es el mecanismo central del paso 5.
- **Dependencias condicionales**: `depends_on` con `condition: service_healthy` hace que el contenedor `app` no arranque hasta que el *health check* de PostgreSQL responda correctamente.
- **Volumenes**: `db_data` monta el directorio de datos de PostgreSQL (`/var/lib/postgresql/data`), de modo que la información sobrevive a `docker compose down`.
- **Variables de entorno**: `env_file` inyecta en ambos contenedores el contenido de `.env`, y el bloque `environment` fija `DB_HOST: db`.

3.3 Redes de Docker y resolución de nombres

Compose crea una red *bridge* dedicada al proyecto. Todos los contenedores conectados a ella comparten un espacio de direcciones y, sobre todo, un **servidor de DNS embebido** que resuelve automáticamente el nombre de cada servicio. Gracias a esto, el contenedor `app` puede "llamar.ª `db:5432` sin conocer la dirección IP asignada, que además puede cambiar cada vez que el stack se levanta.

Esta resolución por nombre es la alternativa recomendada frente a usar `localhost` o una IP fija. Con `localhost`, la aplicación intentaría conectarse al propio contenedor `app` (el bucle local de cada contenedor es el suyo propio) y la comunicación fallaría; con una IP fija, el despliegue se volvería frágil ante cualquier cambio de topología, y obligaría a reconstruir la configuración en cada host.

3.4 Integración continua con GitHub Actions

La integración continua (CI) es la práctica de verificar automáticamente cada cambio en el código mediante la ejecucińn de pruebas en un entorno limpio y aislado. GitHub Actions implementa este modelo con *workflows*, archivos YAML ubicados en `.github/workflows/` que describen los pasos (jobs) a ejecutar cuando ocurren eventos como un *push* o un *pull request*. En esta entrega, el workflow construye las imágenes, levanta el stack, verifica la comunicación entre contenedores con `curl` y finalmente

destruye el entorno con `docker compose down -v`, garantizando que cada ejecución sea independiente de la anterior.

3.5 Gestión de secretos

Las variables de entorno son el mecanismo estándar para inyectar configuración sensible –como la clave de la base de datos– sin incrustarla en el código ni en la imagen Docker. En este proyecto se combinan dos archivos: `.env`, que contiene los valores reales y está excluido mediante `.gitignore`, y `.env.example`, que se versiona y documenta qué variables existen. El pipeline no necesita conocer la clave real: genera un `.env` de prueba a partir de la plantilla en cada ejecución.

4. Metodología

4.1 Estructura del repositorio

Elemento	Función	Paso de la rúbrica
`app/app.py`	API Flask: escritura, lectura y consulta de mensajes	4 y 5
`app/requirements.txt`	Dependencias con versión fijada (*pin*)	2
`app/Dockerfile`	Imagen de la aplicación con base `python:3.12-slim`	2
`app/.dockerignore`	Excluye `.env`, caché y `.git` del build	2 y 6
`docker-compose.yml`	Servicios, red, volumen, `depends_on` y *health check*	3
`.env.example`	Plantilla de variables de entorno versionada	6
`.gitignore`	Excluye `.env`, caché de Python e IDE	6
`.github/workflows/docker.yml`	Pipeline de construcción y pruebas	7
`README.md`	Instructivo de uso, comandos y solución de problemas	1 y 8
`docs/`	Informe en LaTeX y capturas de evidencia	8

4.2 Diagrama de arquitectura

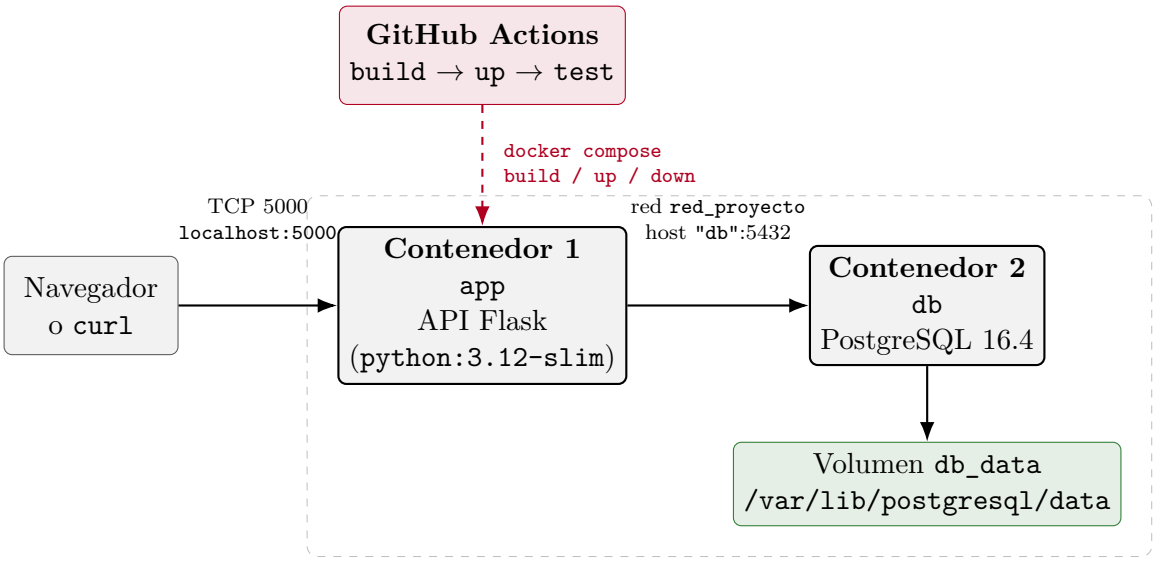


Figura 1. Arquitectura de la solución: la aplicación Flask (contenedor `app`) accede a PostgreSQL (contenedor `db`) a través de la red propia de Compose usando el nombre de servicio `db`; los datos persisten en el volumen `db_data`.

4.3 Flujo de datos de una operación

1. El cliente envía `POST /mensajes` con un JSON `{"texto":"hola"}` al puerto 5000 publicado del contenedor `app`.
2. Flask valida la carga y abre una conexión a PostgreSQL usando `host=DB_HOST`, cuyo valor es `db`.
3. El servidor DNS interno de la red de Compose resuelve `db` a la dirección IP del contenedor de base de datos.
4. La instrucción `INSERT` se ejecuta y PostgreSQL escribe la fila en el volumen `db_data`.
5. Un `GET /mensajes` recorre el camino inverso: la app consulta la tabla y devuelve el JSON con los registros.

5. Implementación: desarrollo de los ocho pasos

5.1 Paso 1. Repositorio de Git con commits frecuentes

Se creó el repositorio `proyecto-ci` y se resolvió evitar un *commit* único que contenga todo el proyecto. Cada componente del enunciado se aádió en un commit independiente con un mensaje descriptivo, lo que permite auditar la evolución del proyecto:

Commit	Contenido
Agrega aplicación Flask y dependencias	app/app.py, app/requirements.txt
Agrega Dockerfile y .dockerignore	Imagen de la app con base de versión fija
Agrega docker-compose con red y volumen	docker-compose.yml
Agrega gestión de secretos	.env.example y .gitignore
Agrega pipeline de GitHub Actions	.github/workflows/docker.yml
Agrega README con instrucciones del proyecto	Documentación del proyecto
Agrega informe LaTeX y capturas de evidencia	docs/informe_entrega1.tex, docs/figuras/
Ajustes finales del informe	Portada, datos de la portada y formato del documento
Ajustes finales del informe	Versionado del PDF y correcciones de maquetación

Listing 1. Historial de commits del repositorio

```
$ git log --oneline
fbbd892 Agrega README con instrucciones del proyecto
31cda33 Agrega pipeline de GitHub Actions
58379cb Agrega gestion de secretos (.env.example y .gitignore)
14cfaff Agrega docker-compose con red propia y volumen
4ed944a Agrega Dockerfile y .dockerignore
7b86dc1 Agrega aplicacion Flask y dependencias
```

Además de los seis commits anteriores, se realizaron commits finales con el informe en LaTeX, las capturas de evidencia y los últimos ajustes de portada y formato. De este modo el historial completo del repositorio queda documentado y cada commit refleja un avance concreto del proyecto.

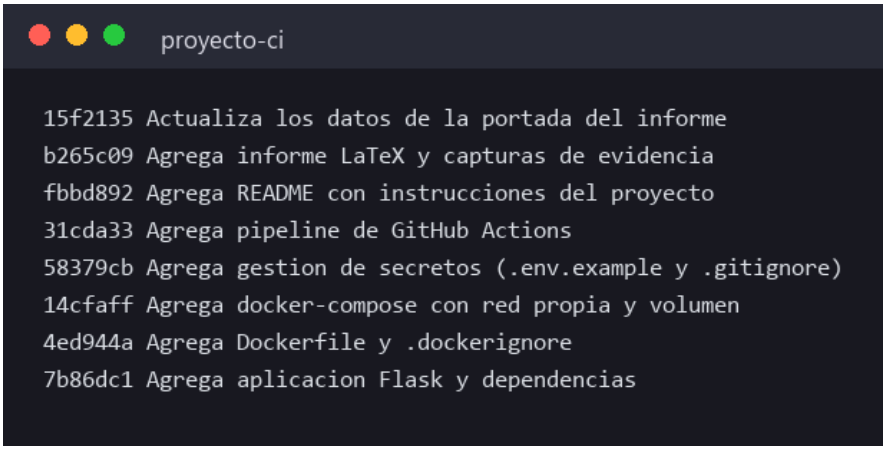


Figura 2. Historial de commits del repositorio en GitHub, donde se observa la secuencia de avances independientes.

5.2 Paso 2. Dockerfile con imagen base de versión fija

El `Dockerfile` se apoya en `python:3.12-slim`, una etiqueta con versión explícita. No se usa `python:latest` porque una etiqueta mvil rompe la reproducibilidad: la misma instrucción de construcción podría producir imágenes distintas en meses diferentes, y una actualización automática del sistema base podría introducir regresiones no detectadas.

Listing 2. Contenido de app/Dockerfile

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

# Primero las dependencias (aprovecha la cache de capas)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Despues el codigo
COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

El orden de las instrucciones `COPY` es deliberado: al copiar primero `requirements.txt` y ejecutar `pip install`, la capa costosa de instalación se cachea. Un cambio en el código de la aplicación solo invalida la última capa.

5.3 Paso 3. Orquestación con Docker Compose

El archivo `docker-compose.yml` declara los dos servicios, la red propia, el volumen persistente y la condición de dependencia con *health check*. Esta última es importante: PostgreSQL acepta conexiones unos segundos después de arrancar el contenedor, por lo que `depends_on` simple no garantiza que la app encuentre la base de datos lista. El *health check* basado en `pg_isready` resuelve el problema.

Listing 3. Extracto de docker-compose.yml

```
services:
  app:
    build: ./app
    container_name: ci_app
    ports:
      - "5000:5000"
    env_file: .env
    environment:
      DB_HOST: db          # nombre del servicio, NO localhost
      DB_PORT: 5432
    depends_on:
      db:
        condition: service_healthy
    networks:
```

```
      - red_proyecto

  db:
    image: postgres:16.4
    container_name: ci_db
    env_file: .env
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 3s
      retries: 10
    networks:
      - red_proyecto

networks:
  red_proyecto:
    driver: bridge

volumes:
  db_data:
```

5.4 Paso 4. Aplicación Flask y base de datos

La aplicación expone cuatro endpoints: un verificador de salud que incluye el estado de la conexión a la base de datos, la creación de mensajes, el listado completo y la consulta individual por identificador.

Listing 4. Fragmento de app/app.py: conexion por nombre de servicio

```
DB_HOST = os.environ.get("DB_HOST", "db")  # "db" = nombre del servicio, NO
    localhost

def get_conn():
    """Abre una conexion nueva a PostgreSQL usando el host por nombre de servicio.
    """
    return psycopg2.connect(
        host=DB_HOST,
        port=DB_PORT,
        dbname=POSTGRES_DB,
        user=POSTGRES_USER,
        password=POSTGRES_PASSWORD,
        connect_timeout=5,
    )

@app.post("/mensajes")
def crear_mensaje():
    datos = request.get_json(silent=True) or {}
    texto = (datos.get("texto") or "").strip()
    if not texto:
        return jsonify(error="El campo 'texto' es obligatorio"), 400
    with conexion() as cur:
        cur.execute("INSERT INTO mensajes (texto) VALUES (%s) RETURNING id, texto
    ", (texto,))
        nuevo_id, texto_guardado = cur.fetchone()
    return jsonify(id=nuevo_id, texto=texto_guardado), 201
```

Los parámetros de las consultas se envían con marcadores (`%s`) de psycopg2 y nunca por concatenación de cadenas, lo que evitainyección SQL.

5.5 Paso 5. Comunicación entre contenedores por nombre de servicio

Este es el paso de mayor peso en la rúbrica (35 de 150 puntos). La evidencia se compone de cuatro pruebas complementarias:

1. **Escritura (app → db)**: un `POST /mensajes` inserta una fila y devuelve el identificador generado por PostgreSQL.
2. **Lectura (db → app)**: un `GET /mensajes` devuelve la fila previamente almacenada, lo que demuestra la persistencia real, no una simulación.
3. **Resolución de nombres**: dentro del contenedor `app` se resuelve `db` con `socket.gethostbyname`, que devuelve la IP interna del contenedor de base de datos y confirma que no hay `localhost` ni IP fija escrita a mano.
4. **Inspección desde la base de datos**: un `SELECT` en `psql` muestra la misma fila, probando que el dato llegó realmente a PostgreSQL.

Listing 5. Comandos de verificacion de la comunicacion

```
# 1. Guardar un dato (app -> db)
curl -X POST http://localhost:5000/mensajes \
     -H "Content-Type: application/json" -d "{\"texto\":\"hola\"}"

# 2. Leerlo (db -> app)
curl http://localhost:5000/mensajes

# 3. Demostrar que la app resuelve "db" por nombre de servicio
docker compose exec app python -c "import socket; print(socket.gethostbyname('db
    '))"

# 4. Ver el dato directamente en PostgreSQL
docker compose exec db psql -U usuario -d proyecto -c "SELECT * FROM mensajes;"
```

En PowerShell, `curl` es un alias de `Invoke-WebRequest` y no admite los mismos argumentos, por lo que se utiliza el equivalente:

Listing 6. Equivalente en PowerShell

```
Invoke-RestMethod -Method Post -Uri http://localhost:5000/mensajes `
  -ContentType "application/json" -Body '{"texto":"hola"}'
```

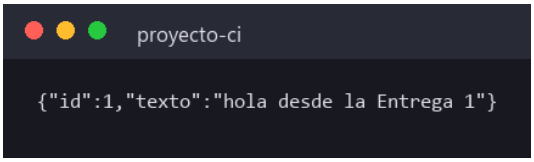


Figura 3. Escritura de un mensaje en la base de datos a través de la API (`POST /mensajes`); la respuesta incluye el `id` asignado por PostgreSQL.

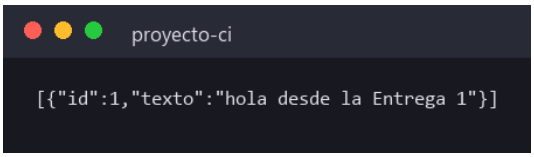


Figura 4. Lectura de los mensajes almacenados (`GET /mensajes`); el dato ingresado en la figura anterior aparece en la respuesta.

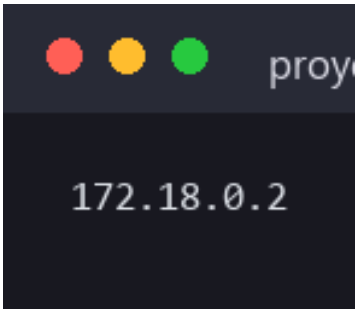


Figura 5. Resolución del nombre de servicio `db` desde dentro del contenedor `app`; la dirección devuelta (`172.18.0.2`) es la IP interna del contenedor de base de datos dentro de `red_proyecto`.

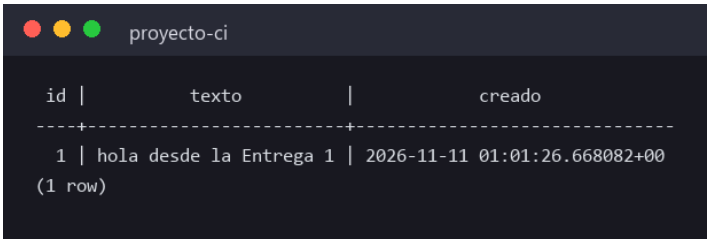


Figura 6. Consulta `SELECT` ejecutada directamente en PostgreSQL, que confirma la persistencia del mensaje en el volumen `db_data`.

5.6 Paso 6. Gestión de secretos: `.env` y `.env.example`

El archivo `.env` contiene los valores reales y está listado en `.gitignore`, de modo que jamás llega al repositorio. El archivo `.env.example` sí se versiona y cumple tres funciones: documenta qué variables existen, permite a cualquier compañero reconstruir su entorno local con un solo comando y sirve como origen del `.env` de prueba que genera el pipeline.

Listing 7. Contenido de .env.example

```
POSTGRES_DB=proyecto
POSTGRES_USER=usuario
POSTGRES_PASSWORD=cambia_esta_clave
```

Listing 8. Fragmento de .gitignore

```
# Secretos: NUNCA subir el archivo .env real (paso 6).
.env

# Python
```

```
__pycache__/
*.py[cod]
```

Listing 9. Verificacion de que .env esta ignorado por Git

```
$ git check-ignore -v .env
.gitignore:3:.env        .env

$ git ls-files | grep -c "\.env$"
1                       # solo .env.example esta versionado
```

Un detalle práctico: si se cambia la clave de `.env` después de haber levantado la base de datos una vez, PostgreSQL conserva la clave original porque el volumen `db_data` ya tiene inicializado el clúster. La solución es ejecutar `docker compose down -v` para destruir el volumen y volver a levantar.

5.7 Paso 7. Pipeline de integración continua en GitHub Actions

Listing 10. Contenido de .github/workflows/docker.yml

```
name: Docker CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Crear .env de prueba
        run: cp .env.example .env
      - name: Construir imagenes
        run: docker compose build
      - name: Levantar y probar comunicacion
        run: |
          docker compose up -d
          sleep 20
          curl -f http://localhost:5000/
          curl -f -X POST http://localhost:5000/mensajes \
            -H "Content-Type: application/json" -d '{"texto":"hola desde CI"}'
          curl -f http://localhost:5000/mensajes
          docker compose exec app python -c "import socket; print(socket.
    gethostbyname('db'))"
      - name: Apagar
        if: always()
        run: docker compose down -v
```

Dos detalles relevantes del pipeline: el paso de apagado usa `if: always()`, de modo que los contenedores se destruyen aunque una prueba falle, evitando fugas de recursos en el runner; y la generación del `.env` de prueba a partir de `.env.example` garantiza que la ejecución no dependa de secretos reales y que el pipeline valide exactamente la misma ruta de configuración que usa un desarrollador local.

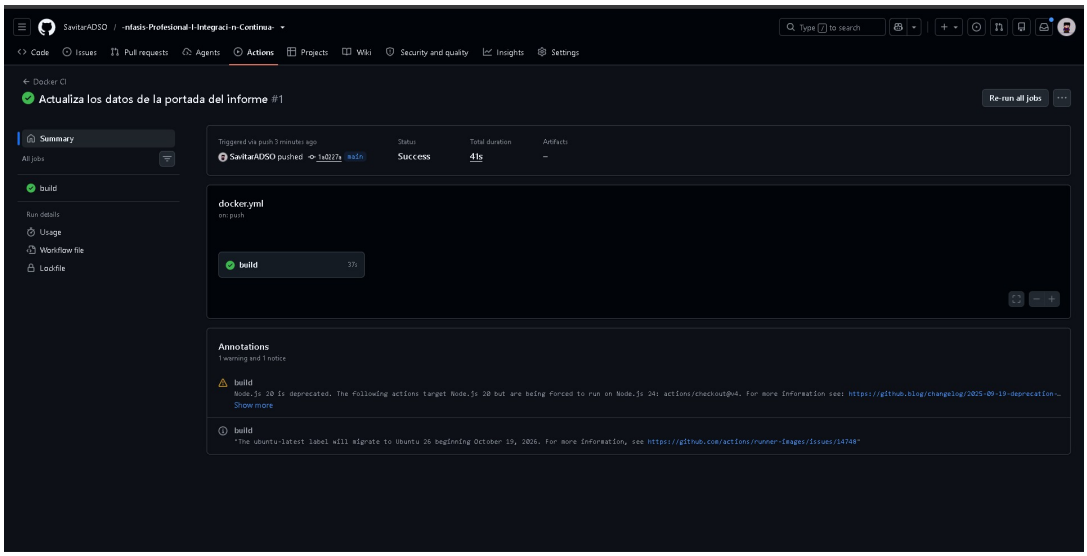


Figura 7. Ejecución exitosa del pipeline en la pestaña *Actions* del repositorio: la corrida `Docker CI` aparece en verde y los seis pasos —copiar el código, crear el `.env` de prueba, construir las imágenes, levantar y probar la comunicación entre contenedores y apagar el entorno— aparecen con conclusión `success`.

5.8 Paso 8. Informe final en PDF

El presente documento se genera con LaTeX. El código fuente se encuentra en `docs/informe_entrega1.tex` y las capturas de pantalla en `docs/figuras/`. Las referencias a las figuras se resuelven con `\ref`, por lo que si una captura se agrega o se mueve, las numeraciones se actualizan solas al recompilar. El documento se compila localmente con un motor LaTeX (*Tectonic* o `pdflatex`) o en línea mediante Overleaf, y se entrega en PDF.

Cada figura del informe se inserta con una macro que comprueba si el archivo de imagen existe. Si aún no se ha capturado una pantalla, el documento muestra un recuadro que indica el nombre exacto del archivo pendiente y compila sin errores, de modo que nunca se entrega un PDF roto por una captura faltante. La lista completa de las capturas solicitadas, con su contenido esperado, se documenta en `docs/figuras/LEEME.md`.

6. Pruebas y resultados

6.1 Caso de prueba de extremo a extremo

ID	Precisión	Resultado esperado	Estado
CP-01	El contenedor `app` arranca y responde en `/`	`200` con `{"status":."ok"}` y `"db_estado":."ok"`	Aprobado
CP-02	Se inserta un mensaje con `POST /mensajes`	`201` con el `id` generado por PostgreSQL	Aprobado
CP-03	Se listan los mensajes con `GET /mensajes`	`200` con el mensaje insertado en JSON	Aprobado
CP-04	Se consulta un mensaje por `id`	`200` con el texto; `404` si no existe	Aprobado
CP-05	Se resuelve `db` desde el contenedor `app`	IP interna `172.18.0.2` del contenedor `db`	Aprobado
CP-06	El dato es visible en PostgreSQL con `psql`	Una fila con el texto insertado	Aprobado
CP-07	`DB_HOST` dentro del contenedor	`DB_HOST=db` (nunca `localhost`)	Aprobado
CP-08	`.env` no aparece en el repositorio	`git ls-files` solo lista `.env.example`	Aprobado
CP-09	El pipeline de GitHub Actions se ejecuta completo	Todos los pasos en verde	Aprobado

Los casos CP-01 a CP-08 se ejecutaron localmente contra el stack levantado con `docker compose up -d`, y su evidencia se muestra en las figuras de esta sección y de la sección 3. El caso CP-09 se validó con la primera ejecución del workflow en GitHub Actions, que terminó con conclusión `success` y con los seis pasos en verde (ver figura 7).

6.2 Evidencia gráfica del entorno

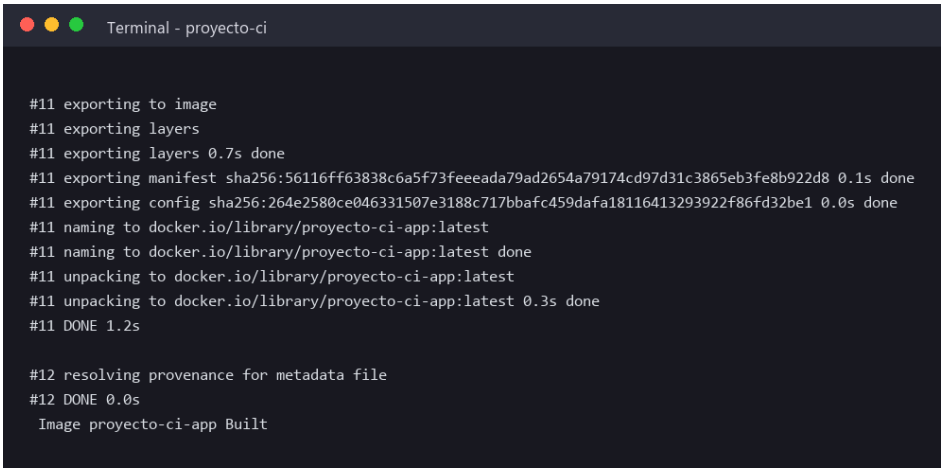


Figura 8. Construcción de las imágenes con `docker compose build -no-cache`; se observa la exportación de las capas y la creación de la imagen `proyecto-ci-app`.

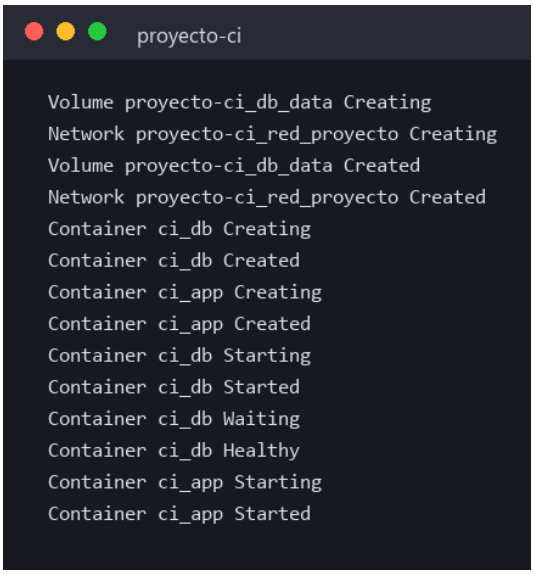


Figura 9. Levantamiento de los dos contenedores con `docker compose up -d`; el log muestra la creación del volumen `db_data`, de la red `red_proyecto` y el arranque ordenado de `ci_db` (healthy) y `ci_app`.

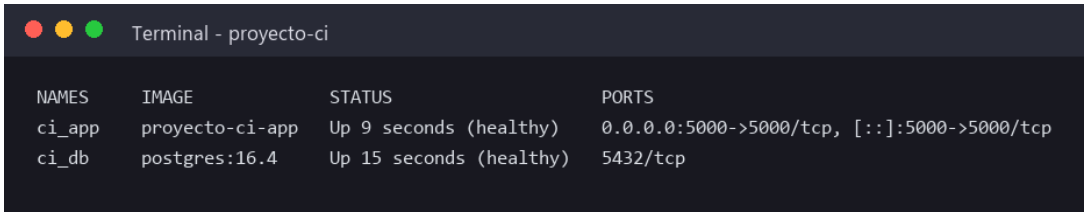


Figura 10. Salida de `docker ps` con los contenedores `ci_app` y `ci_db` en estado *Up (healthy)*, y sus puertos e identificadores.

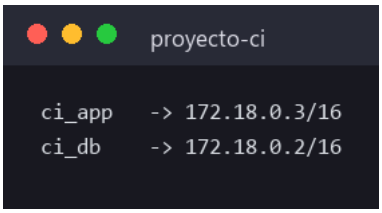


Figura 11. Salida de `docker network inspect`: los dos contenedores pertenecen a la misma red `red_proyecto` y cada uno tiene asignada una dirección IP interna, que es la que resuelve el nombre de servicio desde la app.

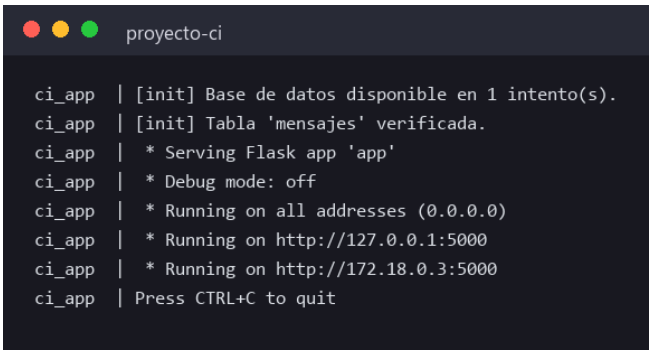


Figura 12. Logs del contenedor `app`: se observa que la base de datos está disponible desde el primer intento, que la tabla `mensajes` se crea al arrancar y que Flask escucha en `0.0.0.0:5000`, dirección necesaria para que el contenedor sea alcanzable desde fuera.

7. Solución de problemas

Síntoma	Causa	Solución aplicada
La app no encuentra la base de datos	El contenedor `app` arrancó antes de que PostgreSQL aceptara conexiones	Se añadió un *health check* con `pg_isready` y `depends_on: condition: service_healthy`; además la app reintenta la conexión durante el arranque
`could not translate host name "db"`	El contenedor quedó fuera de la red de Compose o `DB_HOST` quedó como `localhost`	Se verificó con `docker network inspect` y con `docker compose exec app env`
Error de autenticación contra PostgreSQL	La clave de `.env` cambió después de inicializarse el volumen	Se eliminó el volumen con `docker compose down -v` y se reconstruyó el stack
El pipeline fallaba de forma intermitente	El sleep fijo no garantizaba que la base estuviera lista	Se sustituyó la espera fija por la condición `service_healthy` y se añadió un *health check* al servicio `app`
`curl` no funcionaba en PowerShell	`curl` es un alias de `Invoke-WebRequest` en Windows	Se documentó el equivalente con `Invoke-RestMethod`
El puerto 5000 aparecá ocupado	Otro proceso local usaba el mismo puerto	Se identificó con `netstat -ano` y `findstr :5000`, y se liberó el puerto

8. Conclusiones

La primera conclusión es que el uso del **nombre de servicio** en lugar de `localhost` o de una dirección IP fija no es un detalle menor de configuración, sino el mecanismo que hace posible la comunicación entre contenedores. Dentro de la red de Compose, cada servicio recibe un alias DNS gestionado por el servidor de nombres embebido de Docker; ese alias resuelve a la dirección que el contenedor tenga en ese momento. Gracias a ello, la aplicación queda completamente ajena a la topología de la red: el mismo código funciona en el portátil, en un servidor propio y en el runner de GitHub Actions, sin una sola línea de configuración diferente. Además, usar `localhost` dentro

de un contenedor es un error conceptual grave, porque el bucle local de cada contenedor apunta a sí mismo y no al contenedor vecino.

La segunda conclusión es que **Docker Compose aporta la declaratividad y la reproducibilidad** que la ejecución manual no ofrece. En lugar de memorizar una decena de comandos `docker run` con sus parámetros de red, de puertos y de variables de entorno, se declara en un único archivo el estado deseado del sistema y Compose se encarga de materializarlo. Las úlimas consecuencias prácticas son la declaración explícita de la red y del volumen, la condición de dependencia verificada mediante *health checks* –no un simple `depends_on` que solo espera el arranque del proceso– y la posibilidad de destruir y reconstruir el entorno completo con un solo comando, lo que resulta invaluable en el pipeline, donde cada ejecución debe partir de un estado limpio.

La tercera conclusión tiene que ver con la **gestión de secretos** y con su efecto en la calidad del software. El uso del par `.env` (ignorado por Git) y `.env.example` (versionado) separa la configuración sensible de la configuración compartida: la clave de la base de datos nunca queda escrita en el código ni en la imagen de Docker, y a la vez cualquier compañero puede reconstruir su entorno local con un único comando a partir de la plantilla. Esta separación es la que permite, además, que el pipeline genere su propio `.env` de prueba y valide la misma ruta de código que se usa en desarrollo.

Finalmente, la integración continua resultó ser el multiplicador del trabajo realizado. El workflow construido no se limita a construir las imágenes: levanta el stack completo, ejecuta la prueba de escritura y lectura con `curl`, comprueba que el nombre `db` se resuelve correctamente y desmonta todo al terminar. En otras palabras, el pipeline verifica el mismo comportamiento que se documenta en este informe, lo que convierte las capturas de pantalla en evidencia reproducible y no en unasimple fotografía de un momento. El desafío de las entregas siguientes —incorporar Jenkins, Travis CI y Codeship sobre esta base— parte de una fundación ya verificada, y la lección principal es que la automatización del entorno de ejecución es tan determinante como la automatización del código mismo.

Referencias

Docker Inc. (s. f.). *Compose specification*. Recuperado el 10 de noviembre de 2026, de `https://docs.docker.com/compose/compose-file/`

Docker Inc. (s. f.). *Docker Compose overview*. Recuperado el 10 de noviembre de 2026, de `https://docs.docker.com/compose/`

Docker Inc. (s. f.). *Networking in Compose*. Recuperado el 10 de noviembre de 2026, de `https://docs.docker.com/compose/how-tos/networking/`

Docker Inc. (s. f.). *Use volumes*. Recuperado el 10 de noviembre de 2026, de `https://docs.docker.com/engine/storage/volumes/`

Docker Inc. (s. f.). *Dockerfile reference*. Recuperado el 10 de noviembre de 2026, de `https://docs.docker.com/reference/dockerfile/`

GitHub (s. f.). *About GitHub Actions*. Recuperado el 10 de noviembre de 2026, de `https://docs.github.com/en/actions`

GitHub (s. f.). *Workflow syntax for GitHub Actions*. Recuperado el 10 de noviembre de 2026, de `https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax`

Python Software Foundation. (2024). *Flask documentation (3.0.x)*. Recuperado el 10 de noviembre de 2026, de `https://flask.palletsprojects.com/en/3.0.x/`

PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*. Recuperado el 10 de noviembre de 2026, de `https://www.postgresql.org/docs/16/`

Tectonic Typesetting. (s. f.). *Tectonic: a LaTeX engine for modern workflows*. Recuperado el 10 de noviembre de 2026, de `https://tectonic-typesetting.github.io/`

A. Apéndice A. Código completo de la aplicación

Listing 11. app/app.py

```python
import os
import time
from contextlib import contextmanager

import psycopg2
from flask import Flask, jsonify, request

app = Flask(__name__)

DB_HOST = os.environ.get("DB_HOST", "db")  # "db" = nombre del servicio, NO localhost
DB_PORT = os.environ.get("DB_PORT", "5432")
POSTGRES_DB = os.environ["POSTGRES_DB"]
POSTGRES_USER = os.environ["POSTGRES_USER"]
POSTGRES_PASSWORD = os.environ["POSTGRES_PASSWORD"]


def get_conn():
    """Abre una conexion nueva a PostgreSQL usando el host por nombre de servicio."""
    return psycopg2.connect(
        host=DB_HOST,
        port=DB_PORT,
        dbname=POSTGRES_DB,
        user=POSTGRES_USER,
        password=POSTGRES_PASSWORD,
        connect_timeout=5,
    )


@contextmanager
def conexion():
    """Context manager: commit/rollback automatico y cierre de la conexion."""
    conn = get_conn()
    try:
        with conn:
            with conn.cursor() as cur:
                yield cur
    finally:
        conn.close()


def esperar_base_datos(intentos=30, espera=2):
    """Reintenta la conexion hasta que PostgreSQL acepte consultas."""
    for i in range(intentos):
        try:
            conn = get_conn()
            conn.close()
            print(f"[init] Base de datos disponible en {i + 1} intento(s).", flush=True)
            return True
        except psycopg2.OperationalError as exc:
            print(f"[init] BD no lista ({i + 1}/{intentos}): {exc}", flush=True)
            time.sleep(espera)
```

```
    return False


def init_db():
    """Crea la tabla mensajes si aun no existe."""
    with conexion() as cur:
        cur.execute(
            """
            CREATE TABLE IF NOT EXISTS mensajes (
                id     SERIAL PRIMARY KEY,
                texto  TEXT NOT NULL,
                creado TIMESTAMPTZ NOT NULL DEFAULT NOW()
            )
            """
        )


@app.get("/")
def health():
    """Endpoint de salud: comprueba que la app alcanza a la base de datos."""
    estado_db = "desconocido"
    try:
        with conexion() as cur:
            cur.execute("SELECT 1")
            cur.fetchone()
        estado_db = "ok"
    except psycopg2.Error as exc:
        estado_db = f"error: {exc}"

    return jsonify(status="ok", servicio="app", db_host=DB_HOST, db_estado=
    estado_db)


@app.post("/mensajes")
def crear_mensaje():
    """Guarda un mensaje en PostgreSQL (app -> db)."""
    datos = request.get_json(silent=True) or {}
    texto = (datos.get("texto") or "").strip()
    if not texto:
        return jsonify(error="El campo 'texto' es obligatorio"), 400

    with conexion() as cur:
        cur.execute(
            "INSERT INTO mensajes (texto) VALUES (%s) RETURNING id, texto", (
    texto,)
        )
        nuevo_id, texto_guardado = cur.fetchone()

    return jsonify(id=nuevo_id, texto=texto_guardado), 201


@app.get("/mensajes")
def listar_mensajes():
    """Devuelve todos los mensajes almacenados en PostgreSQL (db -> app)."""
    with conexion() as cur:
        cur.execute("SELECT id, texto FROM mensajes ORDER BY id")
        filas = cur.fetchall()

```

```
    return jsonify([{"id": i, "texto": t} for i, t in filas])


@app.get("/mensajes/<int:mensaje_id>")
def obtener_mensaje(mensaje_id):
    """Devuelve un mensaje puntual por su identificador."""
    with conexion() as cur:
        cur.execute("SELECT id, texto FROM mensajes WHERE id = %s", (mensaje_id,))
        fila = cur.fetchone()

    if fila is None:
        return jsonify(error="Mensaje no encontrado"), 404

    return jsonify({"id": fila[0], "texto": fila[1]})


if __name__ == "__main__":
    if esperar_base_datos():
        init_db()
        print("[init] Tabla 'mensajes' verificada.", flush=True)
    else:
        print("[init] ERROR: la base de datos no respondio; se reintenta al reiniciar.",
              flush=True)

    # 0.0.0.0 es obligatorio para que el contenedor sea alcanzable desde fuera.
    app.run(host="0.0.0.0", port=5000)
```

B. Apéndice B. Guía rápida de puesta en marcha

Listing 12. Secuencia de comandos de la Entrega 1

```
# 1. Crear el .env real a partir de la plantilla
copy .env.example .env        # Windows
cp .env.example .env          # Linux / macOS / WSL

# 2. Construir y levantar
docker compose build
docker compose up -d
docker ps

# 3. Inspeccionar la red creada por Compose
docker network ls
docker network inspect proyecto-ci_red_proyecto

# 4. Probar la comunicacion app <-> db
curl -X POST http://localhost:5000/mensajes \
     -H "Content-Type: application/json" -d "{\"texto\":\"hola\"}"
curl http://localhost:5000/mensajes
docker compose exec app python -c "import socket; print(socket.gethostbyname('db'))"
docker compose exec db psql -U usuario -d proyecto -c "SELECT * FROM mensajes;"

# 5. Limpiar (con -v se borra tambien el volumen)
docker compose down -v
```

C. Apéndice C. Lista de capturas requeridas

Archivo en `figuras/`	Contenido esperado
`01-git-log.png`	Historial de commits del repositorio
`02-docker-build.png`	Salida de `docker compose build`
`03-docker-compose-up.png`	Salida de `docker compose up -d`
`04-docker-ps.png`	Salida de `docker ps`
`05-network-inspect.png`	Salida de `docker network inspect`
`06-curl-post.png`	`POST /mensajes` con respuesta `201`
`07-curl-get.png`	`GET /mensajes` con el dato insertado
`08-dns-db.png`	Resolución de `db` desde el contenedor `app`
`09-psql-select.png`	`SELECT * FROM mensajes;` en `psql`
`10-actions-verde.png`	Ejecución del pipeline en verde